

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1108

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL5*)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:40

Annexure A

Industry Problem Solving Task

Code of Conduct:

I E.Ragul / 192571032 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E. Ragul

Annexure A

Industry Problem Solving Task

1. Smart Banking and Financial Management System

A financial institution develops a **Smart Banking and Financial Management System** to manage customer accounts, fund transfers, loan processing, bill payments, and transaction history. The application follows a layered object-oriented architecture consisting of a presentation layer, service layer, domain layer, and data access layer connected to an OODBMS.

The system uses the **Factory Pattern** to create different account types, the **Strategy Pattern** for loan-interest and payment calculation policies, and the **Observer Pattern** for transaction and fraud alerts. A **Repository Pattern** is used to manage communication with persistent banking objects. The architecture is designed to support future services such as investment management and digital wallets.

Question:

Analyze how object-oriented principles, layered architecture, and design patterns contribute to the scalability, security, flexibility, and maintainability of the banking system. Explain how the application layers communicate with the OODBMS to store and retrieve persistent banking objects.

ANSWER:

1. Presentation / View Layer

- Provides the interface for customers and bank employees.
- Allows users to view accounts, transfer funds, apply for loans, and pay bills.
- Sends user requests to the Service Layer.
- Displays the results returned by the Service Layer.

Example:

Login Page → Account Details → Fund Transfer → Transaction History

2. Service / Business Layer

- Contains the main banking business logic.
- Validates customer requests and performs banking operations.
- Communicates with the Domain Layer and Repository Layer.
- Handles authentication, fund transfers, loan processing, and payments.

Example:

FundTransferService → Check Balance → Validate Account → Create Transaction

3. Domain Layer

This layer represents the main banking objects.

Classes:

- Customer
- Account
- SavingsAccount
- CurrentAccount
- Loan
- Transaction
- Payment

Object-oriented principles are applied here:

- **Encapsulation:** Protects balance, PIN, and account information.
- **Inheritance:** `SavingsAccount` and `CurrentAccount` inherit from `Account`.
- **Polymorphism:** Different account types can implement their own interest calculations.
- **Abstraction:** Complex banking operations are represented through simple methods.

4. Data Access Layer

The Data Access Layer communicates with the **OODBMS** using the Repository Pattern.

Example:

```
AccountRepository
├── save (Account)
├── findById (AccountID)
├── update (Account)
└── delete (Account)
```

The Repository hides database-specific operations from the Service Layer.

Factory Pattern

Creates different types of bank accounts.

```
AccountFactory
  ↓
├── Savings Account
├── Current Account
└── Loan Account
```

This makes adding new account types easier.

Strategy Pattern

Used for different loan-interest and payment calculation methods.

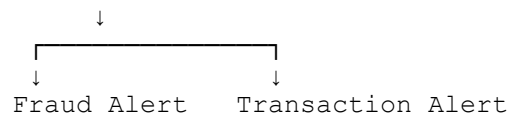
```
Loan
  ↓
Interest Strategy
├── Fixed Interest
├── Variable Interest
└── Special Interest
```

A new calculation method can be added without changing the `Loan` class.

Observer Pattern

Used for transaction and fraud alerts.

Transaction

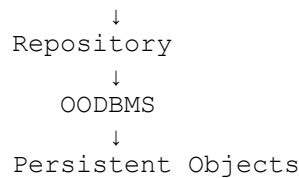


Whenever a transaction occurs, registered observers are notified.

Repository Pattern

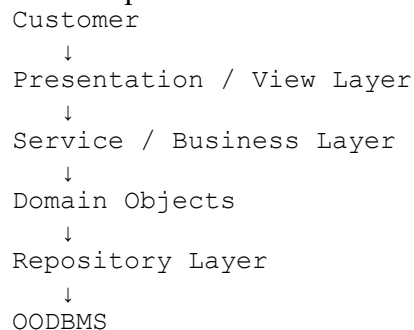
Provides controlled communication between application objects and the OODBMS.

Service Layer



Integration of Access and View Layers with OODBMS

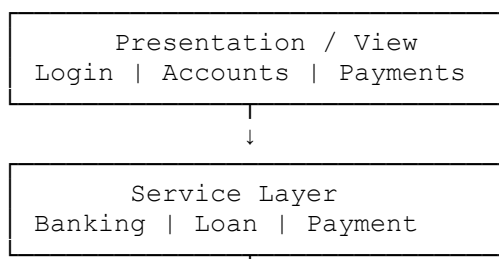
The complete communication flow is:

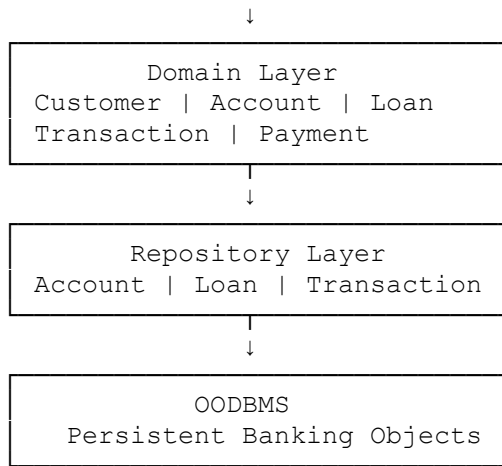


For example, when a customer transfers money:

1. Customer enters transfer details in the **View Layer**.
2. View Layer sends the request to `FundTransferService`.
3. Service Layer validates the account and available balance.
4. Domain objects are updated.
5. `TransactionRepository` stores the transaction.
6. `AccountRepository` updates the account balances in the **OODBMS**.
7. **OODBMS** maintains the objects persistently.
8. Observer objects generate transaction or fraud alerts.
9. The result is returned to the View Layer.
10. The View Layer displays the transaction status to the customer.

Overall Architecture





Conclusion:

The combination of **object-oriented principles, layered architecture, Factory, Strategy, Observer, and Repository patterns** creates a banking system that is **secure, scalable, flexible, and maintainable**. The View and Access layers communicate with the OODBMS through the Service, Domain, and Repository layers, allowing banking objects to be stored and retrieved as persistent objects.

2. Smart Retail and Inventory Management System

A retail organization develops a **Smart Retail and Inventory Management System** to manage products, suppliers, customers, purchase orders, sales transactions, and warehouse stock. The system follows a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system applies the **MVC Pattern** for managing user interfaces, the **Factory Pattern** for creating different product categories, and the **Observer Pattern** to notify managers when inventory reaches a minimum threshold. The **DAO Pattern** handles communication with persistent inventory objects. The architecture allows future integration of automated stock prediction and robotic warehouse management.

Question:

Evaluate how object-oriented design principles, layered architecture, and design patterns improve the flexibility, scalability, and maintainability of the retail system. Explain how the different layers interact with the OODBMS for efficient inventory persistence and retrieval.

ANSWER:

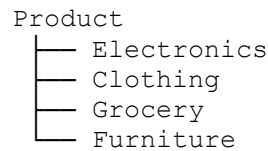
1. Object-Oriented Design Principles

Encapsulation

Product details such as `productID`, `price`, `quantity`, and `supplier` are kept inside classes. Access is controlled through methods such as `addStock()`, `removeStock()`, and `updatePrice()`.

Inheritance

A common `Product` class can be used as a base class. Different product categories can inherit from it.



This reduces code duplication and makes the system easier to extend.

Polymorphism

Different product categories can implement their own behavior. For example, `calculateDiscount()` may work differently for electronics and groceries.

Abstraction

Complex operations such as inventory management, order processing, and database communication are hidden behind simple interfaces.

2. Layered Architecture

The system contains four main layers:

Presentation Layer

Provides the user interface for customers, warehouse staff, and managers.

Examples:

- Product search
- Stock viewing
- Sales entry
- Purchase order creation
- Inventory reports

Business Layer

Contains the business rules of the retail system.

Examples:

- Checking stock availability
- Processing sales
- Updating inventory
- Creating purchase orders
- Applying discounts

Domain Layer

Contains the main retail objects:

- `Product`
- `Supplier`
- `Customer`
- `PurchaseOrder`
- `SalesTransaction`

Data Access Layer

Communicates with the OODBMS using the **DAO Pattern**.

ProductDAO
SupplierDAO
CustomerDAO
InventoryDAO
OrderDAO

The DAO performs operations such as storing, retrieving, updating, and deleting persistent objects.

3. Design Patterns

MVC Pattern

The MVC pattern separates the user interface from business data and processing.

User
↓
View
↓
Controller
↓
Model

- **Model:** Represents products, inventory, customers, and transactions.
- **View:** Displays product and stock information.
- **Controller:** Processes user requests and communicates with the business layer.

This improves maintainability because changes to the interface do not require major changes to business logic.

Factory Pattern

The Factory Pattern creates different product objects.

ProductFactory
↓
├── Electronics
├── Grocery
└── Clothing

For example, when a new product is added, the factory creates the correct product category object.

This makes the system flexible and allows new product categories to be added easily.

Observer Pattern

The Observer Pattern monitors inventory levels.

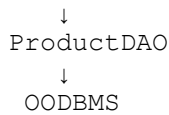
Inventory
↓
Stock Level Changes
↓
Minimum Threshold Reached
↓
Manager Notification

When stock falls below the minimum level, the manager is automatically notified. This can later be extended to automated purchase orders or stock prediction systems.

DAO Pattern

The **Data Access Object (DAO)** provides an interface between the application and the OODBMS.

Business Layer



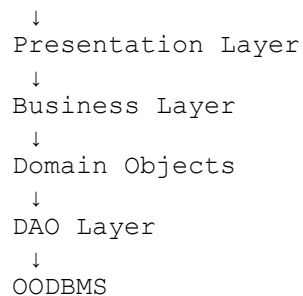
Persistent Product Object

The business layer does not need to know the internal database operations.

4. Interaction with OODBMS

The complete flow is:

User

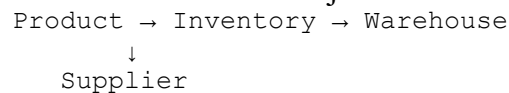


For example, when a product is sold:

1. The customer purchases a product through the **Presentation Layer**.
2. The **Business Layer** checks product availability.
3. The `Product` and `Inventory` domain objects are updated.
4. The `InventoryDAO` stores the updated inventory object.
5. The OODBMS maintains the object persistently.
6. The Observer checks whether stock has reached the minimum threshold.
7. If the threshold is reached, the manager receives a notification.
8. Updated stock information can later be retrieved from the OODBMS through the DAO.

5. Efficient Inventory Persistence and Retrieval

The OODBMS stores objects such as:



The DAO provides operations such as:

```
saveProduct()
findProduct()
updateProduct()
deleteProduct()
```

```
saveInventory()
findInventory()
updateInventory()
```

Because the system uses an OODBMS, objects and their relationships can be stored directly as persistent objects. This simplifies the retrieval of related information such as a product's supplier and warehouse stock.

6. Flexibility

The architecture is flexible because:

- Factory Pattern allows new product categories.
- Observer Pattern allows new notification services.
- MVC allows changes to the user interface independently.
- DAO separates database operations from business logic.
- Inheritance and polymorphism allow different product behaviors.

Future features such as **automated stock prediction** and **robotic warehouse management** can therefore be added with minimal changes to existing components.

7. Scalability

The layered architecture separates responsibilities, allowing individual components to be expanded independently.

For example:

Existing System



Inventory Management



Stock Prediction



Automated Reordering



Robotic Warehouse Management

New services can be added without completely redesigning the system.

8. Maintainability

The system is easier to maintain because each layer has a specific responsibility.

Layer	Responsibility
Presentation	User interface
Business	Retail business rules
Domain	Retail objects and behavior
Data Access	OODBMS communication

If the database implementation changes, mainly the **DAO/Data Access Layer** needs modification. Similarly, changes to the user interface can be handled in the Presentation/MVC layer.

Conclusion

The **Smart Retail and Inventory Management System** uses object-oriented principles, layered architecture, and design patterns to provide a **flexible, scalable, and maintainable** solution. **MVC** separates the interface, **Factory** manages product creation, **Observer** provides automatic low-stock notifications, and **DAO** manages communication with the OODBMS. The layers work together to efficiently store, update, and retrieve persistent inventory objects while supporting future technologies such as **stock prediction and robotic warehouse management**.

3. Smart Hotel and Hospitality Management System

A hotel chain develops a **Smart Hotel Management System** to manage guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. The application uses a layered object-oriented architecture consisting of presentation, service, domain, and data access layers connected to an OODBMS.

The system implements the **Strategy Pattern** for different pricing and discount policies, the **Factory Pattern** for creating room and service objects, and the **Observer Pattern** for reservation and service notifications. A **Repository Pattern** is used to simplify communication between business components and persistent hotel objects. Future enhancements include smart-room automation and personalized guest services.

Question:

Analyze how object-oriented principles and design patterns support flexibility, reusability, and maintainability in the hotel management system. Explain how the layered architecture coordinates reservation processing and communicates with the OODBMS for persistent data management.

ANSWER:

Encapsulation

Guest, room, reservation, and payment details are kept inside classes. Methods such as `bookRoom()`, `checkIn()`, and `makePayment()` control access to the data.

Inheritance

A base `Room` class can be inherited by different room types.



This improves code reuse.

Polymorphism

Different room types can have different implementations of `calculatePrice()`.

Abstraction

Complex reservation, payment, and room-service operations are hidden behind simple methods.

2. Design Patterns

Strategy Pattern

Used for different pricing and discount policies.

```
PricingStrategy
├── RegularPricing
├── SeasonalPricing
└── LoyaltyDiscount
```

It allows new pricing methods to be added without changing the main reservation system.

Factory Pattern

Used to create different room and service objects.

```
RoomFactory
    ↓
StandardRoom
DeluxeRoom
SuiteRoom
```

It makes object creation simple and reusable.

Observer Pattern

Used for reservation and service notifications.

```
Reservation
    ↓
Guest Notification
Reception Notification
Service Notification
```

Whenever a reservation is created or updated, the required users can be notified automatically.

Repository Pattern

Used to communicate between business components and the OODBMS.

```
Service Layer
    ↓
ReservationRepository
    ↓
OODBMS
```

It provides operations such as `save()`, `find()`, `update()`, and `delete()`.

3. Layered Architecture

The system contains four layers:

```
Presentation Layer
    ↓
Service Layer
    ↓
Domain Layer
    ↓
Data Access Layer
    ↓
OODBMS
```

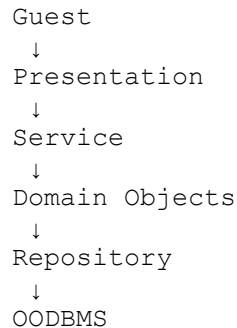
Presentation Layer: Handles guest registration, room booking, payments, and feedback.

Service Layer: Handles business logic such as checking room availability, calculating prices, and processing reservations.

4. Reservation Processing

When a guest reserves a room:

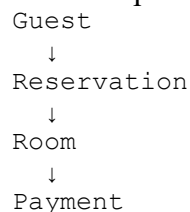
1. Guest enters booking details in the **Presentation Layer**.
2. **Service Layer** checks room availability.
3. The **Domain Layer** creates a `Reservation` object.
4. **Strategy Pattern** calculates the room price and discount.
5. **Repository** stores the reservation in the OODBMS.
6. **Observer Pattern** sends a reservation confirmation.
7. The result is displayed to the guest.



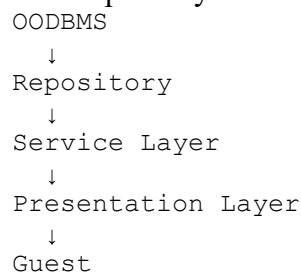
5. OODBMS Communication

The OODBMS stores hotel objects as persistent objects.

For example:



The Repository retrieves these objects when required:



This makes storing and retrieving related hotel information easier.

6. Conclusion

The Smart Hotel Management System uses **encapsulation, inheritance, polymorphism, and abstraction** to improve code reuse and security. The **Strategy, Factory, Observer, and Repository Patterns** provide flexibility and maintainability. The layered architecture separates user interface, business logic, domain objects, and database operations. The Repository communicates with the **OODBMS** to efficiently store and retrieve persistent hotel objects. This structure also supports future features such as **smart-room automation and personalized guest services**.

4. Smart Logistics and Delivery Management System

A logistics company develops a **Smart Logistics and Delivery Management System** to manage shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer notifications. The system adopts a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system uses the **Strategy Pattern** for selecting optimal delivery routes, the **Factory Pattern** for creating different shipment types, and the **Observer Pattern** for real-time delivery status notifications. A **DAO Pattern** manages persistent shipment and vehicle information. The architecture is designed to support future integration with autonomous delivery vehicles and AI-based route optimization.

Question:

Examine how object-oriented design principles, layered architecture, and design patterns improve the adaptability, scalability, and maintainability of the logistics system. Explain how the layers communicate with the OODBMS to manage persistent shipment, vehicle, and delivery information.

ANSWER:

Encapsulation

Shipment, vehicle, and delivery information is stored inside classes. Methods such as `registerShipment()`, `allocateVehicle()`, and `updateStatus()` control access to the data.

Inheritance

A common `Shipment` class can be inherited by different shipment types.

```
Shipment
├── StandardShipment
├── ExpressShipment
└── FragileShipment
```

This improves code reuse.

Polymorphism

Different shipment types can have different delivery costs or handling methods.

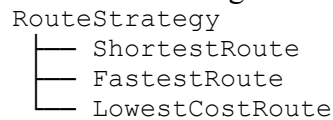
Abstraction

Complex operations such as route selection, vehicle allocation, and delivery tracking are hidden behind simple methods.

2. Design Patterns

Strategy Pattern

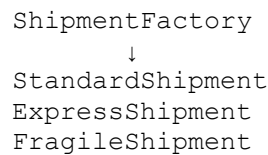
Used for selecting the best delivery route.



The company can change the route strategy without changing the main delivery system.

Factory Pattern

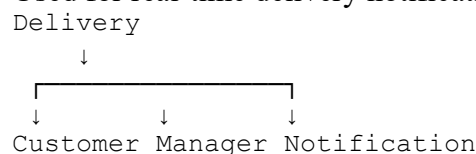
Used to create different shipment objects.



This makes shipment creation flexible and reusable.

Observer Pattern

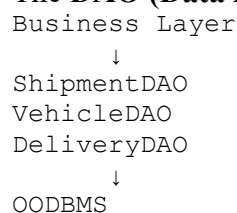
Used for real-time delivery notifications.



Whenever the delivery status changes, registered users are automatically notified.

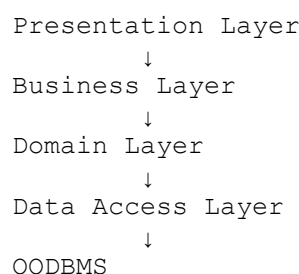
DAO Pattern

The **DAO (Data Access Object)** manages communication with the OODBMS.



DAO provides methods such as `save()`, `find()`, `update()`, and `delete()`.

3. Layered Architecture



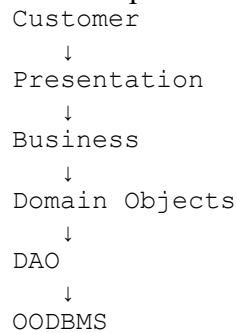
Presentation Layer: Provides shipment registration, tracking, and delivery-status interfaces.

Business Layer: Handles route selection, vehicle allocation, shipment processing, and delivery rules.

4. Delivery Processing

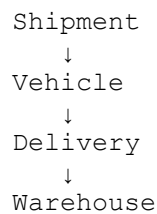
When a customer registers a shipment:

1. Shipment details are entered through the **Presentation Layer**.
2. The **Business Layer** validates the shipment.
3. The **Factory Pattern** creates the appropriate shipment object.
4. The **Strategy Pattern** selects the optimal delivery route.
5. A vehicle is allocated.
6. The **DAO** stores shipment and vehicle information in the OODBMS.
7. The **Observer Pattern** sends delivery-status notifications.
8. Updated delivery information is displayed to the customer.

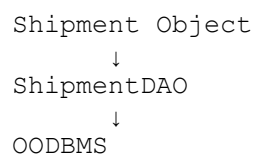


5. OODBMS Communication

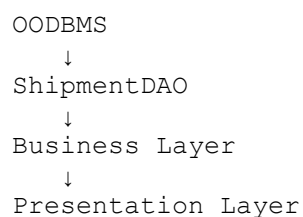
The OODBMS stores persistent logistics objects such as:



Storing Data



Retrieving Data



This allows shipment, vehicle, warehouse, and delivery information to be stored and retrieved efficiently.

6. Adaptability, Scalability and Maintainability

Adaptability: Strategy Pattern allows new route-selection methods, such as AI-based optimization, to be added easily.

Scalability: Factory Pattern allows new shipment types and vehicle types to be introduced without redesigning the complete system.

Maintainability: Layered architecture separates presentation, business logic, domain objects, and database operations, making changes easier to manage.

The system can later support **autonomous delivery vehicles and AI-based route optimization**.

Conclusion

The Smart Logistics and Delivery Management System uses **encapsulation, inheritance, polymorphism, and abstraction** to improve code reuse and organization. The **Strategy, Factory, Observer, and DAO patterns** provide flexibility, scalability, and maintainability. The layered architecture allows the system to process shipments efficiently, while the DAO layer communicates with the **OODBMS** to store and retrieve persistent shipment, vehicle, warehouse, and delivery information.

5. Smart Energy Management System

An energy management company develops a **Smart Energy Management System** to monitor electricity consumption, manage smart meters, analyze energy usage, and generate alerts for abnormal consumption. The system follows a layered object-oriented architecture where the presentation layer provides dashboards, the business layer performs energy analysis, the domain layer manages energy-related rules, and the data access layer communicates with an OODBMS.

The system uses the **Observer Pattern** for consumption alerts, the **Strategy Pattern** for different energy optimization algorithms, and the **Factory Pattern** for creating different smart-meter objects. A **Repository Pattern** is used to manage persistent energy data. The architecture is designed to accommodate future renewable-energy monitoring and AI-based consumption prediction.

Question:

Evaluate how object-oriented principles, layered architecture, and design patterns contribute to the scalability, extensibility, and maintainability of the energy management system. Explain the interaction between the system layers and the OODBMS for storing, retrieving, and managing persistent energy-monitoring objects.

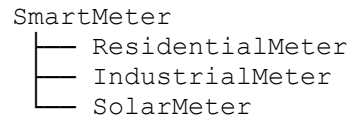
ANSWER:

Encapsulation

Energy data such as meter ID, electricity consumption, voltage, and usage history are stored inside classes. Methods control access to this data.

Inheritance

A common `SmartMeter` class can be inherited by different meter types.



This improves code reuse.

Polymorphism

Different smart meters can implement energy-monitoring or calculation methods differently.

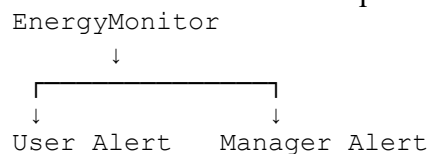
Abstraction

Complex energy analysis and optimization operations are hidden behind simple methods.

2. Design Patterns

Observer Pattern

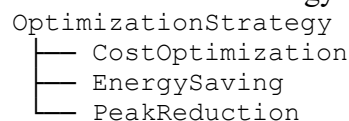
Used for abnormal consumption alerts.



When abnormal electricity consumption is detected, registered users or managers are automatically notified.

Strategy Pattern

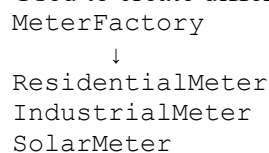
Used for different energy optimization algorithms.



New optimization algorithms can be added without changing the main energy system.

Factory Pattern

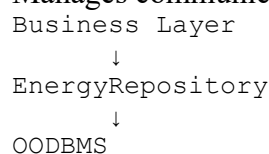
Used to create different smart-meter objects.



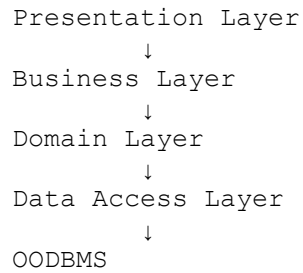
This makes the system flexible when new meter types are introduced.

Repository Pattern

Manages communication between the application and the OODBMS.



3. Layered Architecture



Presentation Layer: Displays energy dashboards, consumption graphs, and alerts.

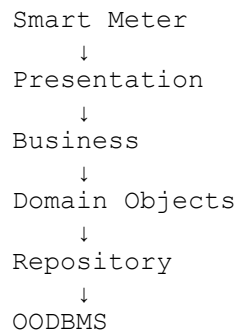
Business Layer: Performs energy analysis, abnormal consumption detection, and optimization.

Domain Layer: Contains objects such as SmartMeter, EnergyReading, EnergyUsage, and Alert.

Data Access Layer: Uses the Repository Pattern to store and retrieve energy objects from the OODBMS.

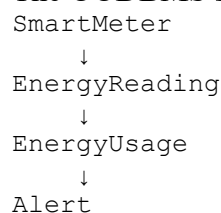
4. Energy Monitoring Process

1. Smart meter collects electricity consumption.
2. The Factory creates the appropriate meter object.
3. The Domain Layer processes the energy reading.
4. The Business Layer analyzes consumption.
5. Strategy Pattern selects an optimization algorithm.
6. If abnormal consumption is detected, Observer Pattern sends an alert.
7. Repository stores the energy data in the OODBMS.
8. The Presentation Layer displays updated information on the dashboard.



5. OODBMS Communication

The OODBMS stores persistent objects such as:



Store Data

```
EnergyReading
  ↓
EnergyRepository
  ↓
OODBMS
```

Retrieve Data

```
OODBMS
  ↓
EnergyRepository
  ↓
Business Layer
  ↓
Dashboard
```

This allows energy-monitoring information to be stored and retrieved efficiently.

6. Scalability, Extensibility and Maintainability

Scalability: The layered architecture allows new energy-monitoring services to be added independently.

Extensibility: Factory and Strategy Patterns allow new smart-meter types and optimization algorithms to be introduced easily.

Maintainability: Repository separates database operations from business logic, while object-oriented principles reduce code duplication.

The system can later support **renewable-energy monitoring** and **AI-based energy-consumption prediction**.

Conclusion

The Smart Energy Management System uses **object-oriented principles, layered architecture, Observer, Strategy, Factory, and Repository Patterns** to provide a scalable, extensible, and maintainable solution. The layers communicate through well-defined responsibilities, while the Repository Layer manages persistent energy objects in the **OODBMS**. This architecture also makes it easier to introduce future technologies such as **renewable-energy monitoring and AI-based consumption prediction**.

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation